Practical-4: Study and implement various operations on Python dictionaries such as creation, insertion, updating, deletion, traversal, searching, copying, and nested dictionaries.

Use a Python dictionary employees to represent employee salary details and perform the following tasks:

1. Populate the dictionary employees with at least 10 employees, where the key is the employee ID and the value is the salary.

```
employees = {
    "1": 50000,
    "2": 60000,
    "3": 70000,
    "4": 80000,
    "5": 90000,
    "6": 100000,
    "7": 110000,
    "8": 120000,
    "9": 130000,
    "10": 140000
}
```

2. Display the complete dictionary.

```
print(employees)
```

```
{'1': 50000, '2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 1
```

3. Display the salary of a specific employee using the employee ID

```
print(employees["1"])
```

```
50000
```

4. Display all employee IDs present in the dictionary.

```
print(employees.keys())
```

```
dict_keys(['1', '2', '3', '4', '5', '6', '7', '8', '9', '10'])
```

5. Display all salary values present in the dictionary.

```
print(employees.values())
```

```
dict_values([50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000, 130000, 140000])
```

6. Find and display the total number of employees using len().

```
print(len(employees))
```

```
10
```

7. Add the salary details of a new employee to the dictionary.

```
employees["11"] = 150000
print(employees)
```

```
{'1': 50000, '2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 1
```

8. Update the salary of an existing employee.

```
employees["1"] = 55000
print(employees)
```

```
{'1': 55000, '2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 1
```

9. Remove an employee record using the del statement.

```
del employees["1"]
print(employees)
```

```
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 
```

10. Remove and display the last inserted employee record using popitem().

```
print(employees.popitem())
```

```
('11', 150000)
```

11. Traverse the dictionary using a for loop to display employee IDs and salaries.

```
for key, value in employees.items():
    print(key, value)
```

```
2 60000
3 70000
4 80000
5 90000
6 100000
7 110000
8 120000
9 130000
10 140000
```

12. Search for a given employee ID and display the corresponding salary.

```
print(employees.get("2"))
```

```
60000
```

13. Check whether a given employee ID exists in the dictionary using the in operator.

```
print("2" in employees)
```

```
True
```

14. Count and display the number of employees whose salary is greater than a specified value.

```
count = 0
for value in employees.values():
    if value > 100000:
        count += 1
print(count)
```

```
4
```

15. Find and display the maximum and minimum salary from the dictionary values.

```
print(max(employees.values()))
print(min(employees.values()))
```

```
140000
60000
```

16. Find and display the total and average salary of all employees.

```
print(sum(employees.values()))
print(sum(employees.values()) / len(employees))
```

```
900000
100000.0
```

17. Create a copy of the dictionary using the copy() method.

```
employees_copy = employees.copy()
print(employees_copy)
```

```
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 1
```

18. Demonstrate the difference between assignment and copying of dictionaries.

```
employees_copy2 = employees.copy()
employees_copy2["1"] = 5000
print(employees_copy2)
print(employees)

employees_copy = employees
```

```
employees_copy["1"] = 55000
print(employees_copy)
print(employees)
```

```
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 1
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 1
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 1
{'2': 60000, '3': 70000, '4': 80000, '5': 90000, '6': 100000, '7': 110000, '8': 120000, '9': 1
```

19. Create a nested dictionary to store employee details such as {ID: {Name, Department, Salary}}.

```
employees_nested = {
    "1": {"Name": "John", "Department": "HR", "Salary": 50000},
    "2": {"Name": "Jane", "Department": "HR", "Salary": 60000},
    "3": {"Name": "Bob", "Department": "HR", "Salary": 70000},
    "4": {"Name": "Alice", "Department": "HR", "Salary": 80000},
    "5": {"Name": "Mike", "Department": "HR", "Salary": 90000},
    "6": {"Name": "Sarah", "Department": "HR", "Salary": 100000},
    "7": {"Name": "Tom", "Department": "HR", "Salary": 110000},
    "8": {"Name": "Emily", "Department": "HR", "Salary": 120000},
    "9": {"Name": "David", "Department": "HR", "Salary": 130000},
    "10": {"Name": "Lisa", "Department": "HR", "Salary": 140000}
}
```

20. Display complete details of all employees from the nested dictionary.

```
print(employees_nested)
```

```
{'1': {'Name': 'John', 'Department': 'HR', 'Salary': 50000}, '2': {'Name': 'Jane', 'Departmen
```

21. Extract and display only employee names from the nested dictionary.

```
print([value["Name"] for value in employees_nested.values()])
```

```
['John', 'Jane', 'Bob', 'Alice', 'Mike', 'Sarah', 'Tom', 'Emily', 'David', 'Lisa']
```

22. Extract and display only salary values from the nested dictionary.

```
print([value["Salary"] for value in employees_nested.values()])
```

```
[50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000, 130000, 140000]
```

23. Convert all salaries into a new dictionary containing only employees with salary above the average salary.

```
greater_than_average = {key: value for key, value in employees.items() if value > sum(employe
print(greater_than_average)
```

```
{'6': 100000, '7': 110000, '8': 120000, '9': 130000, '10': 140000}
```

24. Use the get() method to safely access the salary of a given employee ID.

```
print(employees.get("2"))
```

```
60000
```

25. Compare dictionaries with lists and tuples with respect to data organization and access efficiency.

```
salary_dict = {"1": 50000, "2": 60000, "3": 70000}
salary_list = [50000, 60000, 70000]
salary_tuple = (50000, 60000, 70000)

print("Dictionary access by key:", salary_dict["1"])
print("List access by index:", salary_list[0])
print("Tuple access by index:", salary_tuple[0])

print("Searching 60000 in dictionary:", 60000 in salary_dict.values())
print("Searching 60000 in list:", 60000 in salary_list)
print("Searching 60000 in tuple:", 60000 in salary_tuple)

salary_dict["4"] = 80000
salary_list.append(80000)

print("Updated Dictionary:", salary_dict)
print("Updated List:", salary_list)
```

```
Dictionary access by key: 50000
List access by index: 50000
Tuple access by index: 50000
Searching 60000 in dictionary: True
Searching 60000 in list: True
Searching 60000 in tuple: True
Updated Dictionary: {'1': 50000, '2': 60000, '3': 70000, '4': 80000}
Updated List: [50000, 60000, 70000, 80000]
```